

PEDIDO

- Crie um código que mostre com um vídeo o transporte dos fluidos do motor, onde os fluidos são representados por linhas que indicam sua temperatura e sugira um site que possa rodar o programa.

CÓDIGO

```
import matplotlib.animation as animation
import matplotlib.pyplot as plt
import numpy as np
from IPython.display import HTML

# Configuração da figura e do cenário do motor
fig, ax = plt.subplots(figsize=(10, 6))
ax.set_xlim(-2, 12)
ax.set_ylim(-2, 8)
ax.axis("off") # Oculta as bordas do gráfico para focar no diagrama
fig.patch.set_facecolor("#111111") # Fundo escuro para destacar o
fluido
ax.set_facecolor("#111111")

# 1. Desenho esquemático dos componentes do motor
# Bloco do Motor
ax.add_patch(
    plt.Rectangle(
        (3, 2), 4, 3, color="#333333", ec="#555555", lw=2, label="Bloco
do Motor"
    )
)
ax.text(
    5,
    3.5,
    "BLOCO DO\nMOTOR",
    color="white",
    fontsize=12,
    weight="bold",
    ha="center",
    va="center",
)

# Radiador
ax.add_patch(
    plt.Rectangle(
        (9, 1.5),
        1.5,
```

```

        4,
        color="#222222",
        ec="#00aaff",
        lw=2,
        label="Radiador",
    )
)
ax.text(
    9.75,
    3.5,
    "RA\nDIA\nDOR",
    color="#00aaff",
    fontsize=10,
    weight="bold",
    ha="center",
    va="center",
)

# Reservatório/Bomba
ax.add_patch(
    plt.Circle((1, 1.5), 0.8, color="#222222", ec="#00ffcc", lw=2,
    label="Bomba")
)
ax.text(
    1,
    1.5,
    "BOMBA",
    color="#00ffcc",
    fontsize=9,
    weight="bold",
    ha="center",
    va="center",
)

# 2. Definição do caminho das linhas de fluido (Tubulações)
# Caminho do fluido: Bomba -> Motor (esquenta) -> Radiador (esfria) ->
Retorno
x_path = [1, 3, 7, 9, 9.75, 9.75, 9, 7, 3, 1, 1]
y_path = [1.5, 2.5, 2.5, 2.5, 2.5, 4.5, 4.5, 4.5, 4.5, 4.5, 1.5]

# Mapeamento de temperatura ao longo do caminho (0 = Frio, 1 = Quente)
# O fluido esquenta dentro do motor (entre x=3 e x=7 na ida) e esfria
no radiador

```

```
temperaturas_base = [0.1, 0.2, 0.9, 1.0, 1.0, 0.6, 0.4, 0.2, 0.1, 0.1, 0.1]
```

```
# Interpolação para criar um caminho suave e contínuo
```

```
passos = 200
```

```
x_fino = np.interp(np.linspace(0, len(x_path) - 1, passos),  
range(len(x_path)), x_path)
```

```
y_fino = np.interp(np.linspace(0, len(y_path) - 1, passos),  
range(len(y_path)), y_path)
```

```
temp_fina = np.interp(  
    np.linspace(0, len(temperaturas_base) - 1, passos),  
    range(len(temperaturas_base)),  
    temperaturas_base,  
)
```

```
# Criando os segmentos multicoloridos (Gradiente de Temperatura)
```

```
# Azul = Frio, Vermelho = Quente
```

```
cmap = plt.get_cmap("jet")
```

```
for i in range(passos - 1):  
    ax.plot(  
        x_fino[i : i + 2],  
        y_fino[i : i + 2],  
        color=cmap(temp_fina[i]),  
        lw=6,  
        alpha=0.3,  
    )
```

```
# 3. Configuração das partículas pulsantes (as linhas que se movem)
```

```
num_particulas = 15
```

```
linhas_fluxo = [  
    ax.plot([], [], "o", markersize=8, markeredgecolor="w")[0]  
    for _ in range(num_particulas)  
]
```

```
# Função de Inicialização da Animação
```

```
def init():  
    for linha in list(linhas_fluxo):  
        linha.set_data([], [])  
    return linhas_fluxo
```

```

# Função de Atualização de cada Quadro (Frame) do Vídeo
def update(frame):
    for idx, linha in enumerate(linhas_fluxo):
        # Defasa as partículas para que fiquem distribuídas ao longo do
tubo
        ponto_atual = (frame + idx * (passos // num_particulas)) %
passos

        xi = x_fino[ponto_atual]
        yi = y_fino[ponto_atual]
        cor_temp = cmap(temp_fina[ponto_atual])

        linha.set_data([xi], [yi])
        linha.set_color(cor_temp)

    return linhas_fluxo

# Barra de Legenda de Temperatura
sm = plt.cm.ScalarMappable(cmap=cmap, norm=plt.Normalize(vmin=40,
vmax=100))
sm.set_array([])
cbar = fig.colorbar(sm, ax=ax, orientation="horizontal", pad=0.05,
shrink=0.6)
cbar.set_label("Temperatura do Fluido (°C)", color="white",
fontsize=10)
cbar.ax.xaxis.set_tick_params(color="white")
plt.setp(plt.getp(cbar.ax.xaxis, "ticklabels"), color="white")

# Gerando a animação (Vídeo de 120 frames)
ani = animation.FuncAnimation(
    fig, update, frames=120, init_func=init, interval=50, blit=True
)

plt.close() # Impede que o matplotlib plote uma imagem estática
indesejada

# Transforma a animação em um vídeo HTML5 player para rodar no site
HTML(ani.to_html5_video())

```

RESULTADO

https://drive.google.com/file/d/1ZmNefAA2xSeJ1HhbEP0hu1RVldMiYe8I/view?usp=drive_link